MERN Deployment

These instructions are for Linux, specifically Ubuntu 22 was used.

First, you need a Digital Ocean account. You should already have this, but you can either remove mysql, php, and apache2 or you can create a new droplet (the latter is preferred).

You will need to point your domain to the droplet IP address.

Create a Linux droplet. **Don't create a LAMP droplet**, just create from the main Digital Ocean dashboard.

I am using ssh to login as shown here. (I had to install SSH for my Windows 10 machine, Windows 11 and Macs have it built in.) I might use Putty from my server.

```
:\Users\Rick>ssh root@198.199.81.74
root@198.199.81.74's password:
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-113-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Mon Aug 12 18:36:51 UTC 2024

  System load:  0.0               Processes:             95
  Usage of /:   6.8% of 24.05GB   Users logged in:       1
  Memory usage: 22%               IPv4 address for eth0: 198.199.81.7
  Swap usage:   0%                IPv4 address for eth0: 10.10.0.6

Expanded Security Maintenance for Applications is not enabled.

37 updates can be applied immediately.
27 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Mon Aug 12 18:35:17 2024 from 35.145.105.96
root@ubuntu-s-1vcpu-1gb-amd-nyc1-01:~#
```

Before you build, you need to change the address of your fetch paths. Here are mine:
http://cop4331-5.com:5000/api/login
http://cop4331-5.com:5000/api/searchcards
http://cop4331-5.com:5000/api/addcard

You should use a conditional and write a function that supports local and remote as follows:

For now, until we get to MERN C, place this in login.tsx and cardui.tsx, thus violating the **DRY** principle.

```
const app_name = 'cop4331-5.com';
function buildPath(route:string) : string
```

```
{
    if (import.meta.env.MODE != 'development')
    {
        return 'http://' + app_name + ':5000/' + route;
    }
    else
    {
        return 'http://localhost:5000/' + route;
    }
}
```

Change the fetch in Login.tsx to the following

```
const response = await fetch(buildPath('api/login'),
     {method:'POST',body:js,headers:{'Content-Type':
'application/json'}});
```

Change the fetches in CardUI.tsx to the following

```
const response = await fetch(buildPath('api/addCard'),
     {method:'POST',body:js,headers:{'Content-Type':
'application/json'}});
```

and

```
const response = await fetch(buildPath('api/searchCards'),
     {method:'POST',body:js,headers:{'Content-Type':
'application/json'}});
```

In the cards directory install dotenv
sudo npm install dotenv

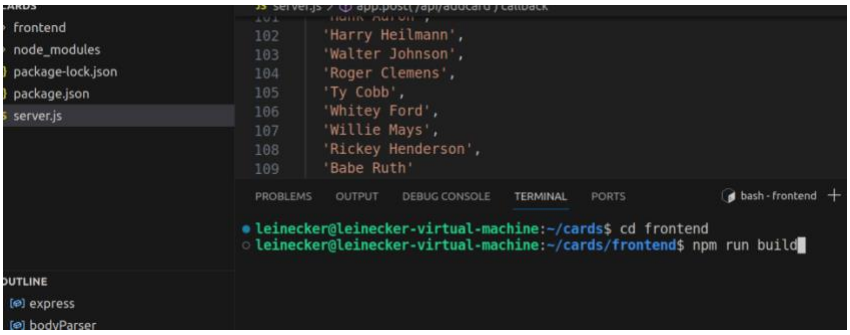
Add a .env file in the cards directory
touch .env

Add your connect string
MONGODB_URI=Your connect string here

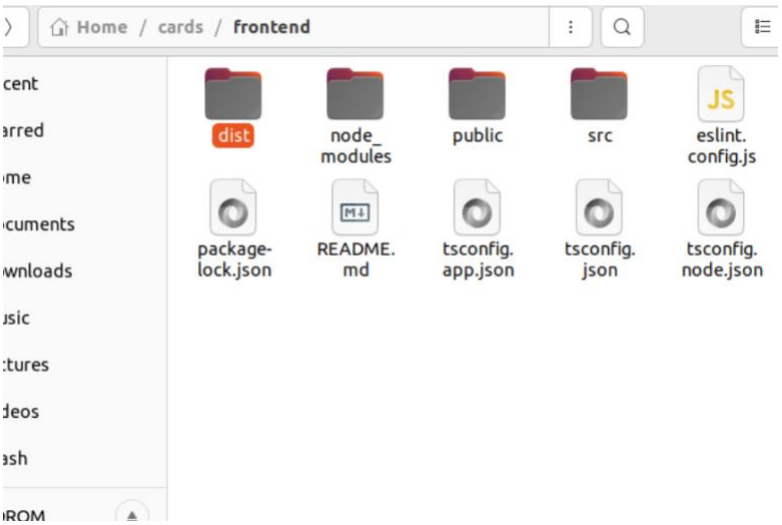
You need to add .env to .gitignore as this file will contain your database connect string and cannot be uploaded to GitHub

Now use this instead of the hard-coded connection string in server.js
Change:
const url = "Your connect string here"
To:
require('dotenv').config();
const url = process.env.MONGODB_URI;

Now you need to **build** the webapp. From the frontend directory in a terminal type **npm run build**.



You can find the build inside of the frontend folder in a folder named dist.



Use SSH to log on to the server.

On the server:
sudo apt-get update
sudo apt-get install nodejs
sudo apt-get install npm
sudo npm cache clean -f
sudo npm install -g n
sudo n stable
sudo apt-get install nginx

Navigate to **/etc/nginx/sites-available**
Open **default** with a text editor.
Edit as follows (make sure to use your domain)

```
server {
        listen 80;
        listen [::]:80;

        root /var/www/html;
        index index.html;

        server_name cop4331-5.com www.cop4331-5.com 209.38.68.57;
```

```
        location / {
                try_files $uri $uri/ /index.html;;
        }
}
```

Go to your domain registrant and set your domain name to point to this server.

Filters Actions

	Type	Name	Data	TTL	Delete	Edit
☐	A	@	198.199.81.74	10800 seconds		
☐	NS	@	ns01.domaincontrol.com.	1 Hour	Can't delete	Can't edit
☐	NS	@	ns02.domaincontrol.com.	1 Hour	Can't delete	Can't edit
☐	CNAME	autoconfig	cop4331-3.com.	10800 seconds		
☐	CNAME	autoconfig.admin	cop4331-3.com.	10800 seconds		

Navigate to /var/www/html
Delete the default html file.
Create index.html as follows:

```
<html>
        <body>
                <h1>We love COP 4331</h1>
        </body>
</html>
```

With a browser use the domain name or the IP address

Not secure cop4331-3.com

We Love COP 4331

Now you need to copy the build files to the server.

```
leinecker@leinecker-virtual-machine:~/cards/frontend/dist$ cd ..
leinecker@leinecker-virtual-machine:~/cards/frontend$ scp -r ./dist/* root@198
199.81.74:/var/www/html
root@198.199.81.74's password:
index-Do9QM3Z3.js                             100%  158KB 686.0KB/s   00:00
index-DiwrgTda.css                            100% 1391    30.6KB/s   00:00
index.html                                    100%  464    10.5KB/s   00:00
vite.svg                                      100% 1497    17.1KB/s   00:00
leinecker@leinecker-virtual-machine:~/cards/frontend$
```

To clarify, my command line is: **scp -r ./dist/* root@cop4331-5.com:/var/www/html** (then you will need to enter your password).

Reboot the server.

Go back to your browser and refresh

Unfortunately, you cannot login because there is no API server listening. Let's do that next.

First, we will create a directory on the server and then copy our server files into it.

From /var to the following:
mkdir cardsServer
cd cardsServer

Now the path to your cards server is **/var/cardsServer** (remember that Linux is case sensitive).

We will not copy the **node_modules** directory since it is very large and we can re-install it on the server.

From the **cards** directory:
scp -r server.js root@cop4331-5.com:/var/cardsServer
scp -r package.json root@cop4331-5.com:/var/cardsServer
scp -r package-lock.json root@cop4331-5.com:/var/cardsServer
scp -r .env root@cop4331-5.com:/var/cardsServer
navigate to /var/cardsServer and type **npm install** (this rebuilds the node_modules directory)
edit package.json and replace **nodemon** with **node**

Navigate to /var/cardsServer
Run **npm install -g pm2**
Run **pm2 start server.js --name "express-server"**
Run **pm2 startup**
Run **pm2 save**

The front end cannot contact the back end because the front end is actually running on your local machine and trying to use port 5000 on the back end server. Open port 5000 with **sudo ufw allow 5000**

CI/CD

CI/CD

GitHub Secrets Setup

First, you need to add these secrets to your GitHub repository:

1. Go to your repository → Settings → Secrets and variables → Actions

2. Add the following secrets:
    - SERVER_HOST: cop4331-3.com
    - SERVER_USER: root
    - SERVER_PASSWORD: cop4331IsCool
    - SERVER_PORT: 22 (default SSH port)

GitHub Actions Workflow File

Create this file in your repository at .github/workflows/deploy.yml:

```
name: Deploy MERN Application

on:
  push:
    branches: [ main ]

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
    # Checkout the repository code
    - name: Checkout code
      uses: actions/checkout@v3

    # Setup Node.js
    - name: Setup Node.js
      uses: actions/setup-node@v3
      with:
        node-version: '22.18'
        cache: 'npm'
        cache-dependency-path: |
          package-lock.json
```

```
      frontend/package-lock.json

# Install and build frontend
- name: Install frontend dependencies
  working-directory: ./frontend
  run: npm ci

- name: Build frontend
  working-directory: ./frontend
  run: npm run build

# Deploy frontend to server
- name: Deploy frontend to server
  uses: appleboy/scp-action@v0.1.5
  with:
    host: ${{ secrets.SERVER_HOST }}
    username: ${{ secrets.SERVER_USER }}
    password: ${{ secrets.SERVER_PASSWORD }}
    port: ${{ secrets.SERVER_PORT }}
    source: "frontend/dist/*"
    target: "/var/www/html"
    strip_components: 2
    rm: true

# Deploy backend files to server
- name: Deploy backend files
  uses: appleboy/scp-action@v0.1.5
  with:
    host: ${{ secrets.SERVER_HOST }}
    username: ${{ secrets.SERVER_USER }}
    password: ${{ secrets.SERVER_PASSWORD }}
```

```
    port: ${{ secrets.SERVER_PORT }}
    source: "server.js,package.json,package-lock.json"
    target: "/var/cardsServer"
    overwrite: true

# Install backend dependencies and restart PM2
- name: Install dependencies and restart server
  uses: appleboy/ssh-action@v1.0.0
  with:
    host: ${{ secrets.SERVER_HOST }}
    username: ${{ secrets.SERVER_USER }}
    password: ${{ secrets.SERVER_PASSWORD }}
    port: ${{ secrets.SERVER_PORT }}
    script: |
      # Navigate to backend directory
      cd /var/cardsServer

      # Install dependencies
      npm ci --production

      # Ensure PM2 is installed globally
      npm list -g pm2 || npm install -g pm2

      # Reload the application with PM2 (zero-downtime reload)
      pm2 reload express-server --update-env || pm2 start server.js --name "express-server"

      # Save PM2 process list
      pm2 save

      # Optional: Clear nginx cache if needed
      nginx -s reload
```

One-Time Server Setup Instructions

1. Connect to your server

bash

ssh root@cop4331-3.com

2. Ensure Node.js 22.18 is installed

bash

 # Check current version

node --version

 # If not 22.18, install it using NodeSource repository

curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -

sudo apt-get install -y nodejs

3. Install PM2 globally (if not already installed)

bash

npm install -g pm2

4. Create necessary directories

bash

 # Ensure backend directory exists

mkdir -p /var/cardsServer

 # Ensure frontend directory exists with proper permissions

mkdir -p /var/www/html

chown -R www-data:www-data /var/www/html

chmod -R 755 /var/www/html

5. Setup PM2 to start on system boot

bash

pm2 startup systemd

 # Follow the command it outputs

pm2 save

6. Ensure .env file is in place

Create or verify your .env file exists in /var/cardsServer/.env with your environment variables. **Note**: The .env file should NOT be in your GitHub repository for security reasons.

Example .env content:

PORT=5000

MONGODB_URI=your_mongodb_connection_string

Add other environment variables as needed

7. Verify Nginx configuration

bash

 # Test nginx configuration

nginx -t

 # If successful, reload nginx

systemctl reload nginx

8. Verify firewall settings

bash

 # Check if port 5000 is open

ufw status

 # If not open, add it

ufw allow 5000/tcp

9. Set proper permissions for deployment

bash

 # Ensure the deployment user can write to these directories

chmod -R 755 /var/cardsServer

chmod -R 755 /var/www/html

10. Initial PM2 setup (if app not already running)

bash

cd /var/cardsServer

 # Only if the app isn't already running

pm2 start server.js --name "express-server"

pm2 save